第五章 传输层



- 传输层概述
- TCP/IP体系中的传输层
- UDP协议
- TCP协议

5.1 传输层概述

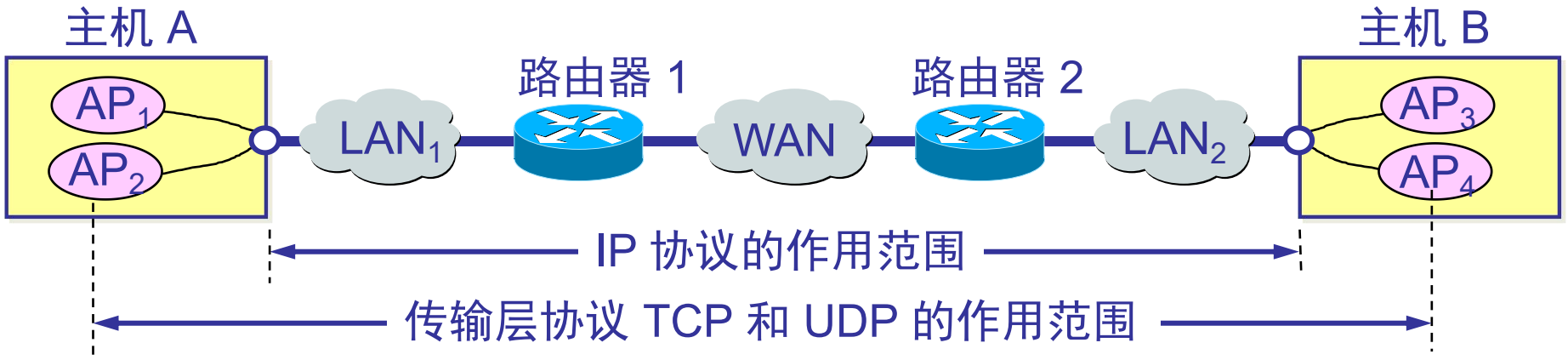


- 传输层位于模型的第四层，向上面的应用层提供通信服务，它属于面向通信部分的最高层，同时也是用户功能中的最低层。

- 只有位于网络边缘部分的主机的协议栈才有传输层，而网络核心部分中的路由器在转发分组时都只用到下三层的功能。

5.1 传输层概述

- 应用进程之间的通信
  - 两个主机进行通信实际上就是两个主机中的应用进程互相通信。
  - 传输层的一个很重要的功能就是复用和分用
  - 应用层不同进程的报文通过不同的端口向下交到传输层，再往下就共用网络层提供的服务



5.1 传输层概述



◊ 网络层和传输层的区别

1. 传输层为应用进程提供端到端（进程到进程）的逻辑通信，网络层为终端主机提供计算机到计算机的逻辑通信；
2. 传输层传送的是包含整个文件的报文，网络层传输的是报文分组，是文件碎片；
3. 传输层对整个报文数据进行差错检验，网络层只对报文头进行差错检验；
4. 网络层使用IP地址，传输层使用插口地址（套接字）
5. 传输层提供面向连接的TCP服务和无连接的UDP服务，网络层只能提供无连接的IP数据报服务。

5.2 TCP/IP体系中的传输层



- 传输层中的两个协议
  - 面向连接的TCP：Transmission Control Protocol，传输控制协议
    - 传输控制协议(Transmission Control Protocol)
    - 在不可靠的网络上，在端到端之间提供一条全双工的可靠的逻辑信道（无丢失、重复、错序、比特错误等）。
    - 传送的数据单位协议是 TCP 报文段(segment)
  - 面向无连接的UDP（User Datagram Protocol）
    - 用户数据报协议
    - 传送的数据单元是 UDP 报文或用户数据报
    - 在端到端之间提供一条不可靠的逻辑信道

5.2 TCP/IP体系中的传输层



- 传输层向高层屏蔽了下面通信子网的细节，它建立的逻辑通信使两个进程看见的就是在两个传输实体之间有一条端到端的逻辑通信信道。
  - 对于TCP服务，这条通信信道是一条全双工的可靠管道，只能进行一对一传输；
  - 对于UDP服务，这是一个连接源端和目的端的不可靠的逻辑通信网，可以进行一对一、一对多传输。

5.2 TCP/IP体系中的传输层



- 说明：
  - 为TCP服务的IP数据报是无连接的，是不可靠的，用网络来传输数据的；TCP得到的数据传输服务必然存在丢失、错序、重复等错误；接收端的TCP服务进程需要纠正所有的这些错误，然后再向上层交付数据
  - 对于接收进程，收到的数据序列与发送的数据序列完全相同，这些数据系列宛如通过一个管道传过来的
  - UDP的不可靠不是说目的端收到的数据有错（比特错），而是说数据包有丢失、重复、错序等问题，需要目的端应用进程解决或能够容忍。传输层对接收数据进行检验，出错的数据不会交给进程。
  - 有比特错的数据单元经数据链路层或UDP检验后被丢弃。

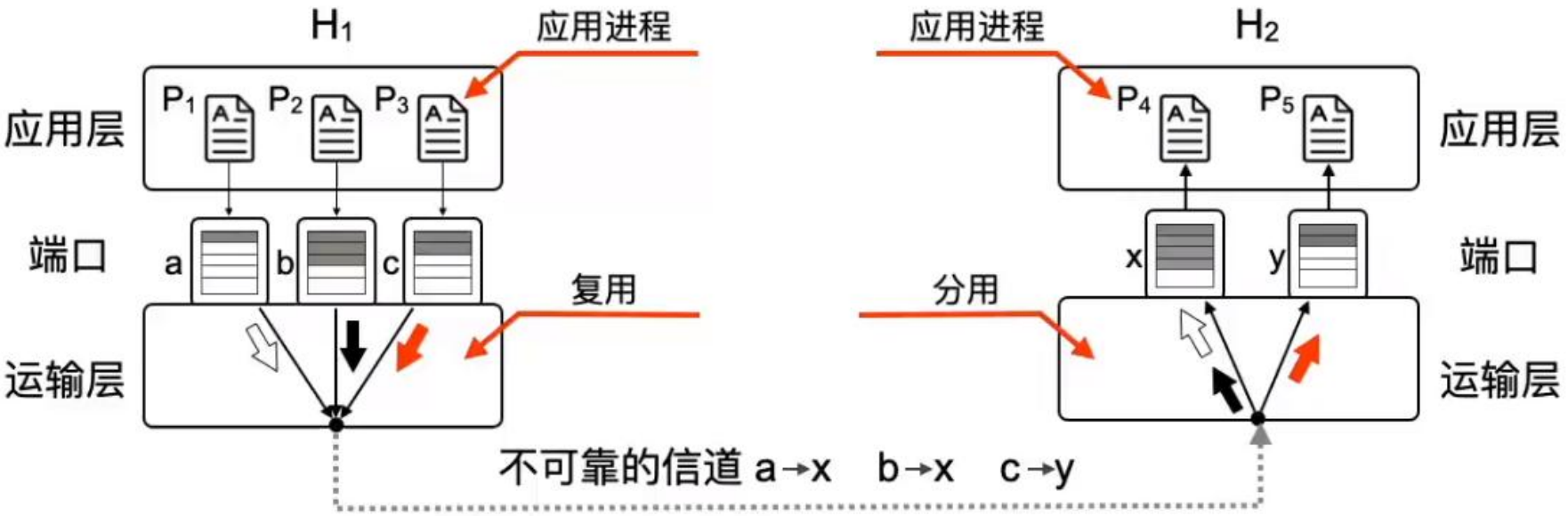
5.2 TCP/IP体系中的传输层



- 传输层地址
  - 传输层地址=IP地址+端口号
    - 又称为插口，套接字。
    - 如：插口=（163.43.23.13，15000）
  - IP地址确定主机在网络中的位置，端口号与主机的进程号相对应。IP地址+端口号就确定了进程在网络上的哪一台计算机上的哪个端口上
  - 端口是计算机的一种资源，由操作系统管理，端口是操作系统分配的数据流通道
  - 端口用16比特标识，端口号有0~65535，其中0~1023为专用端口（熟知端口号），用于网络上的专用服务进程，不分配给一般用户进程

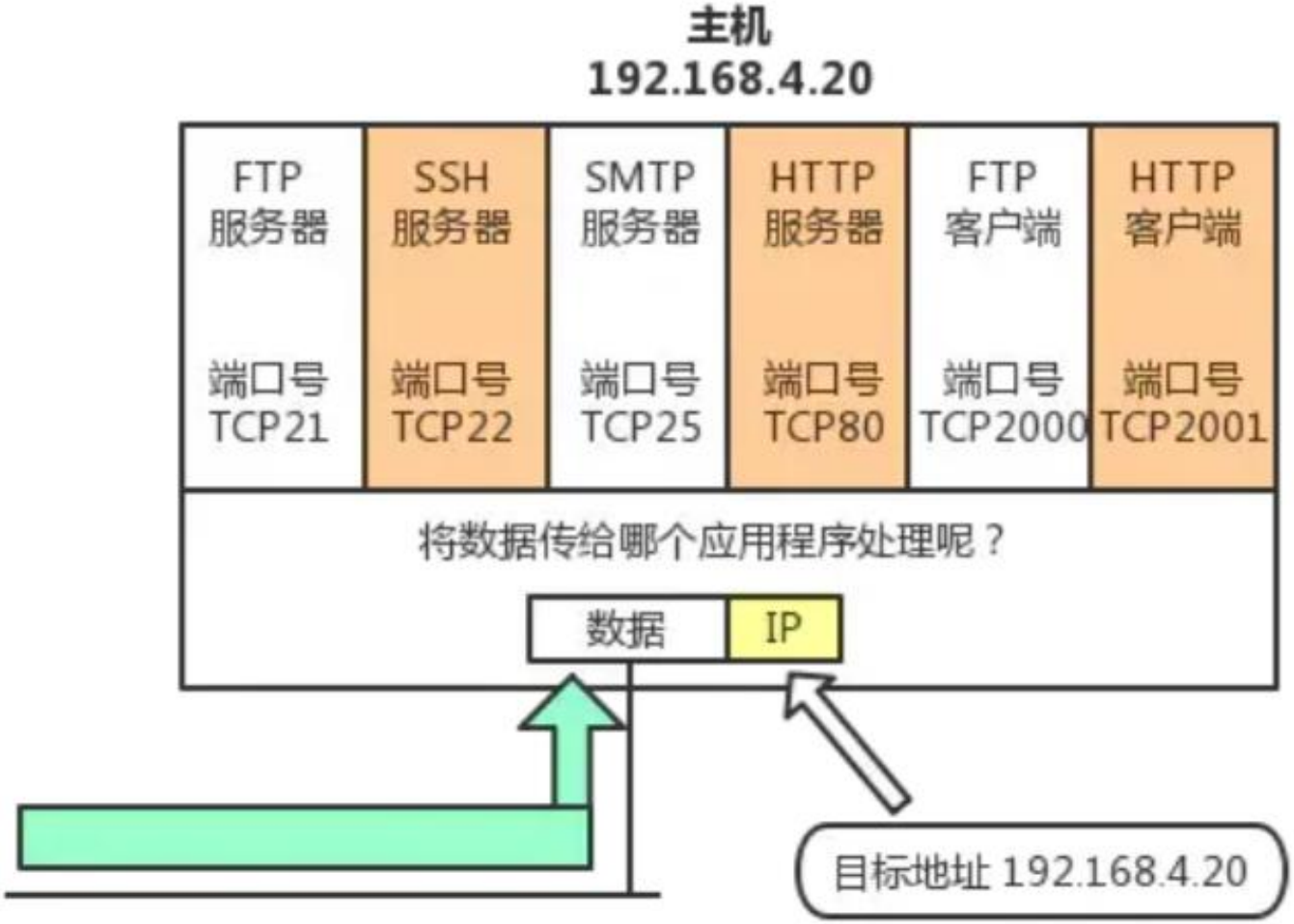
5.2 TCP/IP体系中的传输层

- 端口号
  - 通过端口号识别应用进程
    - 一个应用进程可以有多个端口
    - 一个端口只能属于一个应用进程
  - 通过（IP地址+端口号+协议）识别通信
    - 同一个进程内部可以并行多组通信



请注意：运输层之间的这条虚线不是一条连接

5.2 TCP/IP体系中的传输层

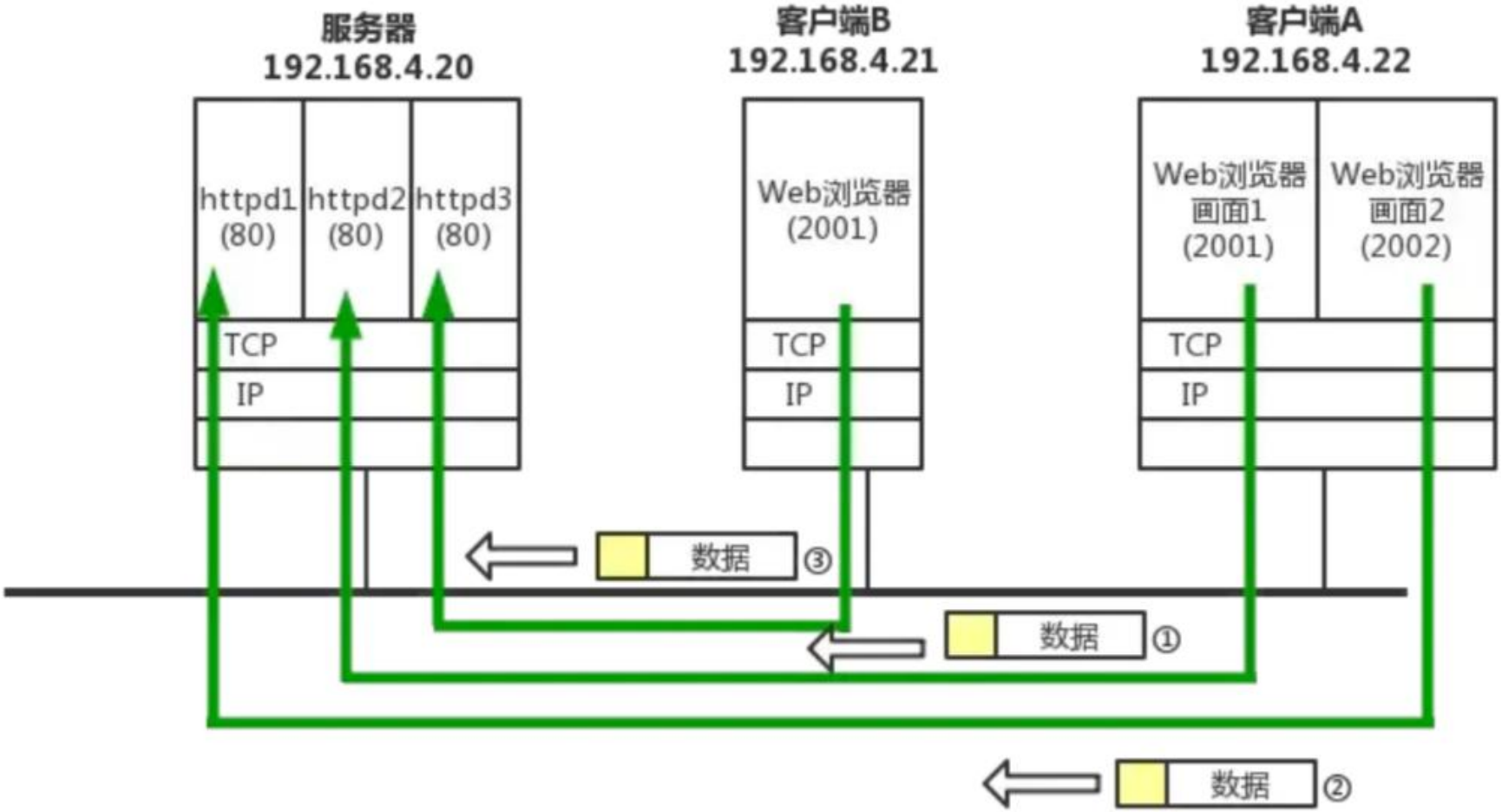


5.2 TCP/IP体系中的传输层



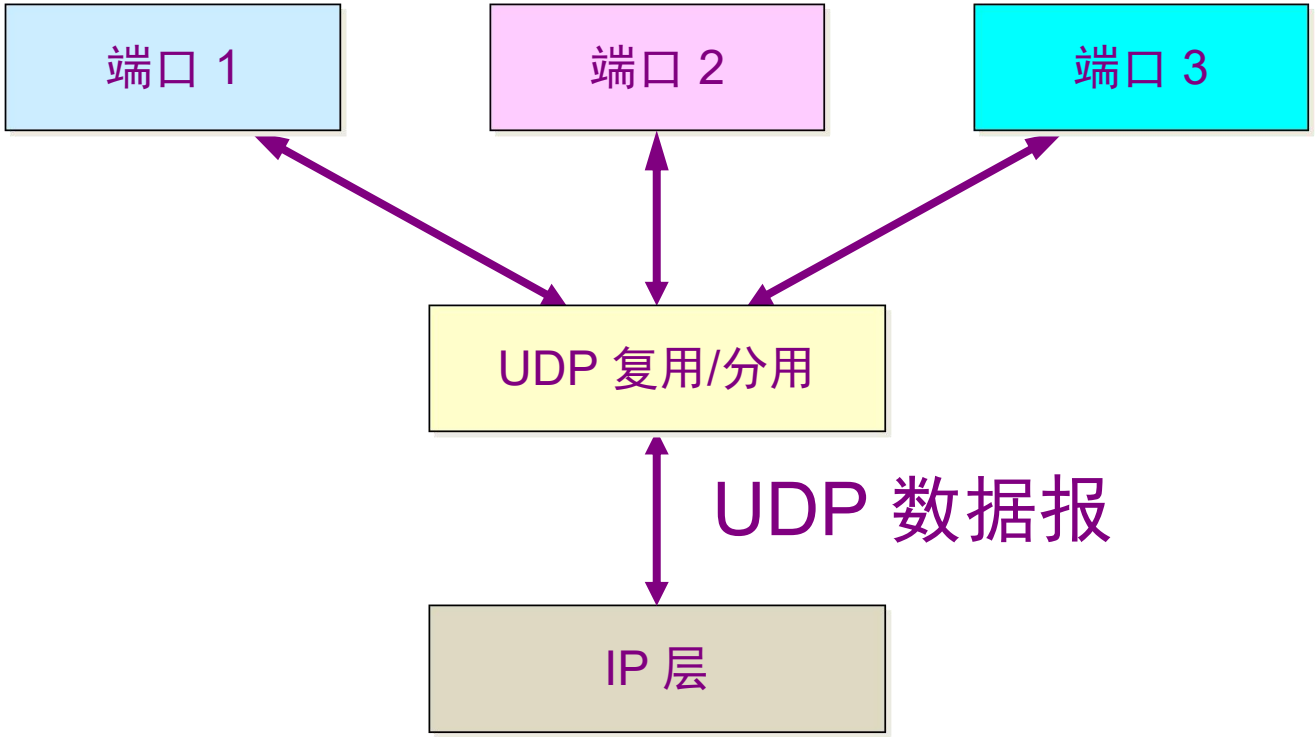
通过源 IP 地址、目标 IP 地址、协议号、源端口号以及目标端口号这五个元素识别一个通信

5.2 TCP/IP体系中的传输层



5.3 UDP协议

- UDP只是在IP数据报服务之上增加了很少一点功能，即复用与分用的功能和差错检验功能。



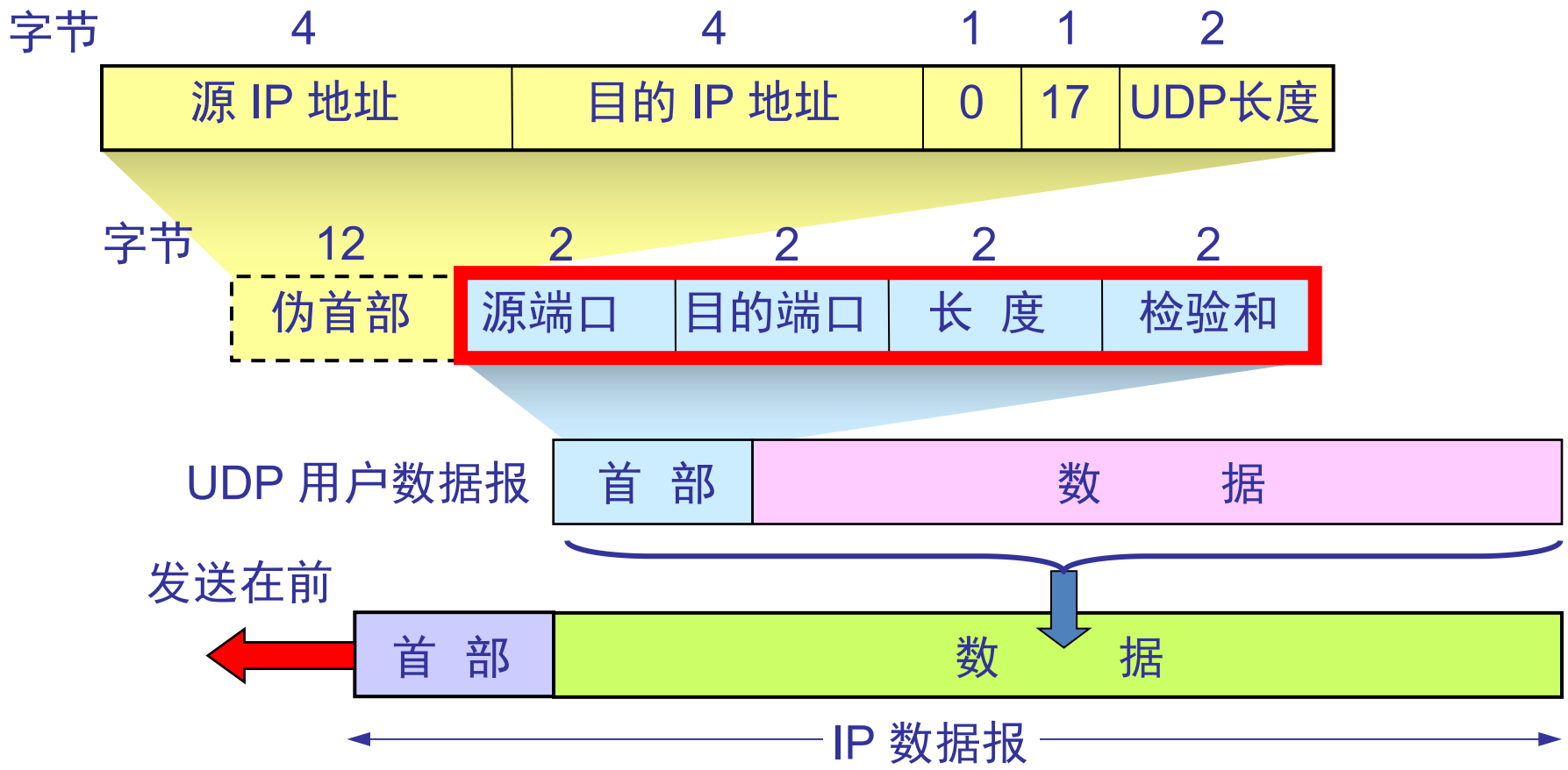
UDP的主要特点



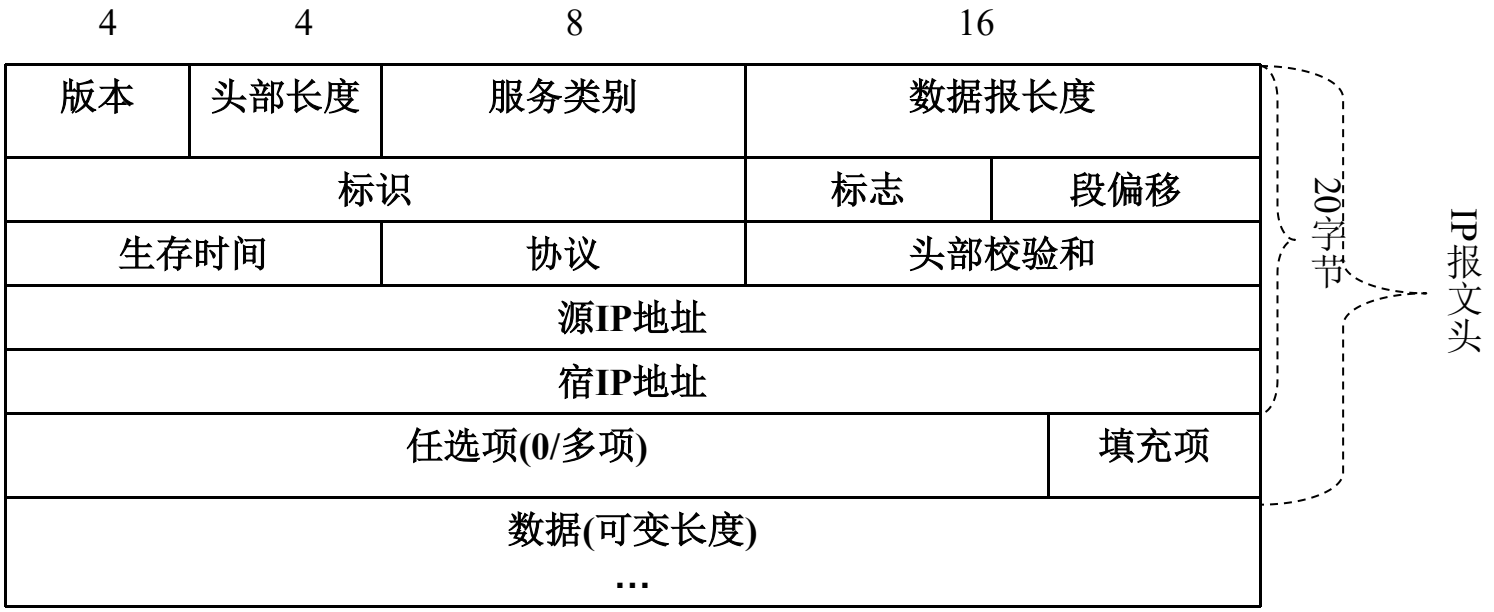
1. UDP属于无连接的传输协议，在IP的数据报服务之上增加了端口的功能。
2. UDP使用尽最大努力交付，不保证可靠交付，不需要维持连接状态，不对数据做任何处理。
3. UDP是面向报文的，对进程交下来的数据添加首部后直接交给IP层，由IP数据报进行传输。即对进程报文不组合、不拆分。进程报文的大小由进程负责考虑。
4. UDP不考虑拥塞控制问题，只以自己的速度发送数据，UDP的数据传输速率是匀速的。
5. UDP支持一对一、一对多、多对一、多对多的交互通信。
6. UDP首部开销小，只有8个字节。

UDP的首部

◊ UDP用户数据报只有两个字段：数据字段和首部字段。 其中首部只有8个字节。



回顾IP报文格式



- 上表所示是IP协议所规定的数据包格式
- 其中，数据项是运输层所交付的要传递的数据。
- 报文头是网络层为传递数据所加的各种控制信息，又称为数据包首部

UDP的首部

- 首部字段包含：源端口字段、目的端口字段、UDP数据报长度字段、检验和字段。
- 伪首部：并不是UDP数据报真正的首部，只是在计算检验和时，临时和UDP数据报连接在一起，得到一个过渡的UDP数据报来计算的。
- 伪首部既不向下传送，也不向上递交。

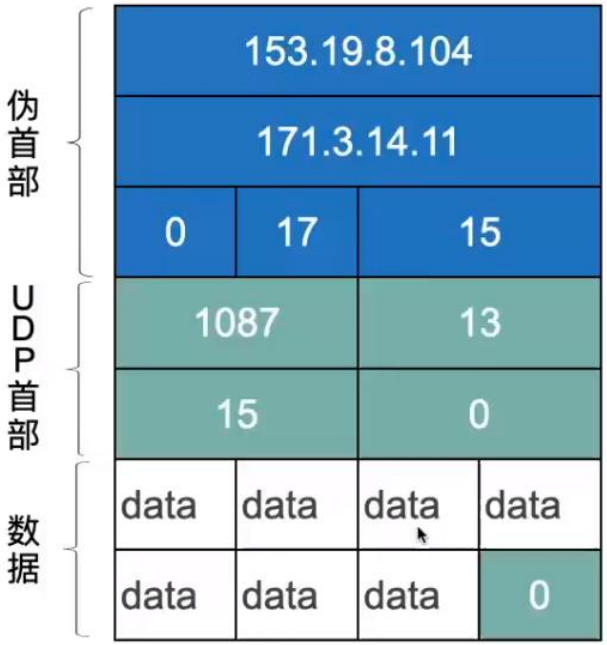
UDP的校验方法



UDP的校验方法

- 检验和字段是对包括伪首部在内的所有部分（伪首部+首部+数据字段）进行检验的结果。
- 检验和字段对数据部分进行检验，这一点不同于IP数据报的检验和字段。
- 接收端通过检验和发现错误，则舍弃UDP整个数据报。

计算UDP检验和的例子



二进制反码运算求和
将得出的结果求反码

10011001	00010011	153.19：39187
00001000	01101000	8.104：2152
10101011	00000011	171.3：43779
00001110	00001011	14.11：3595
00000000	00010001	0和17：17
00000000	00001111	15：15
00000100	00111111	1087：1087
00000000	00001101	13：13
00000000	00001111	15：15
00000000	00000000	0（检验和）：0
01010100	01000101	data：21573
01010011	01010100	data：21332
01001001	01001110	data：18766
01000111	00000000	data和0（填充）：18176
10	10010110	11101011 → 10010110111101101
		取反
		检验和 → 0110100100010010

UDP服务的优点



1. 发送前不建立连接，减少了开销和发送前的时延；
2. 不使用拥塞控制，也不保证可靠交付，因此主机不需要维持许多参数和状态表；
3. 首部只有8个字节，附加信息少；
4. 没有拥塞控制，不会因为拥塞降低数据发送速率，适用于视频会议等实际应用。

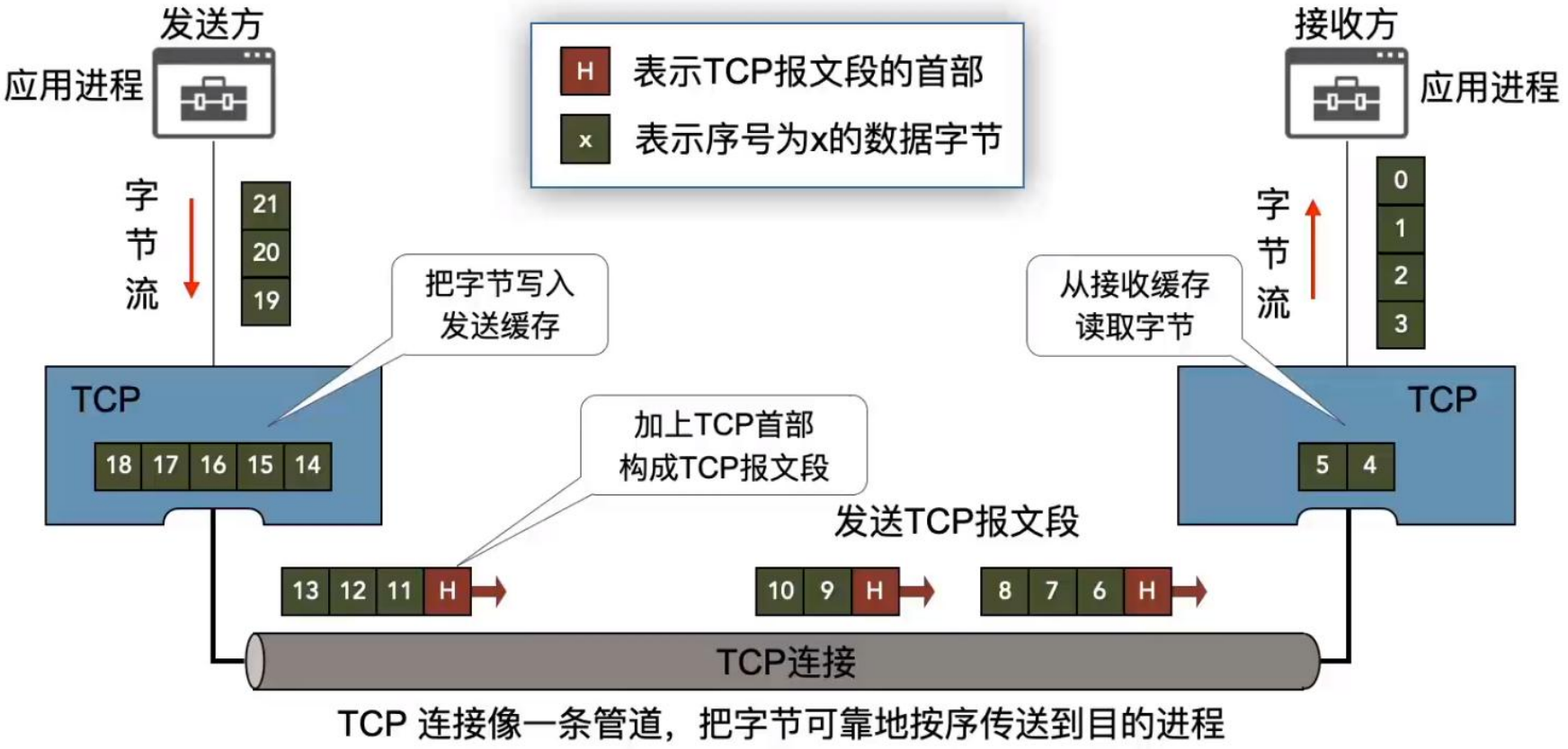
5.4 TCP协议



◊ TCP的特点：

1. TCP服务是面向连接的，有建立连接、传输数据、释放连接三步。
2. 每一条TCP连接只能有两个端点，即是端点对端点连接。
3. TCP服务是可靠的，传输的数据序列无差错、无错序、不丢失、不重复。
4. TCP管道提供全双工通信。
5. TCP传输是面向字节流的。即把数据块看成无结构的字节流，并将字节流以报文段为单位传输。

TCP面向流的概念



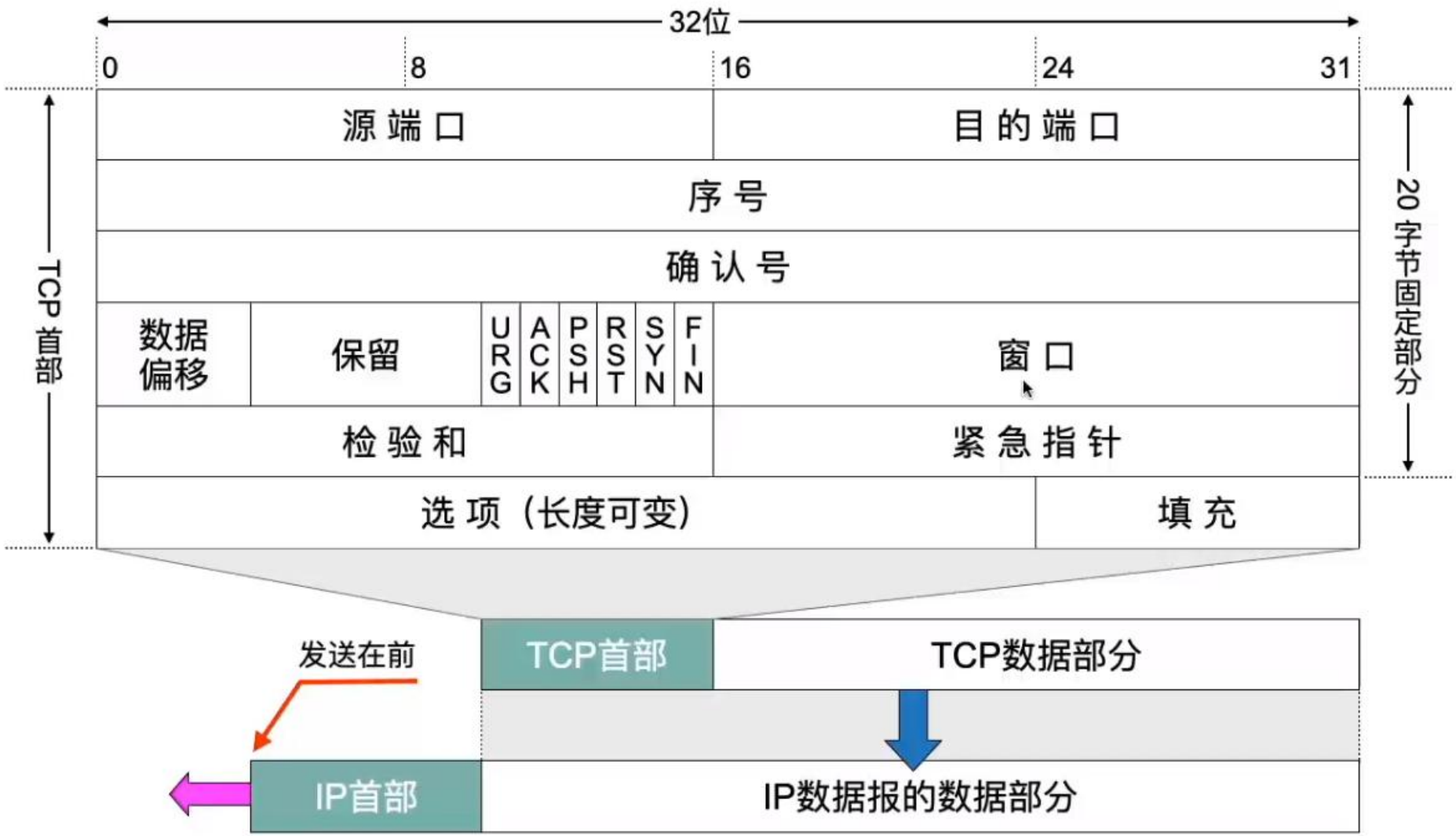
TCP 连接像一条管道，把字节可靠地按序传送到目的进程

5.4.1 TCP报文段格式

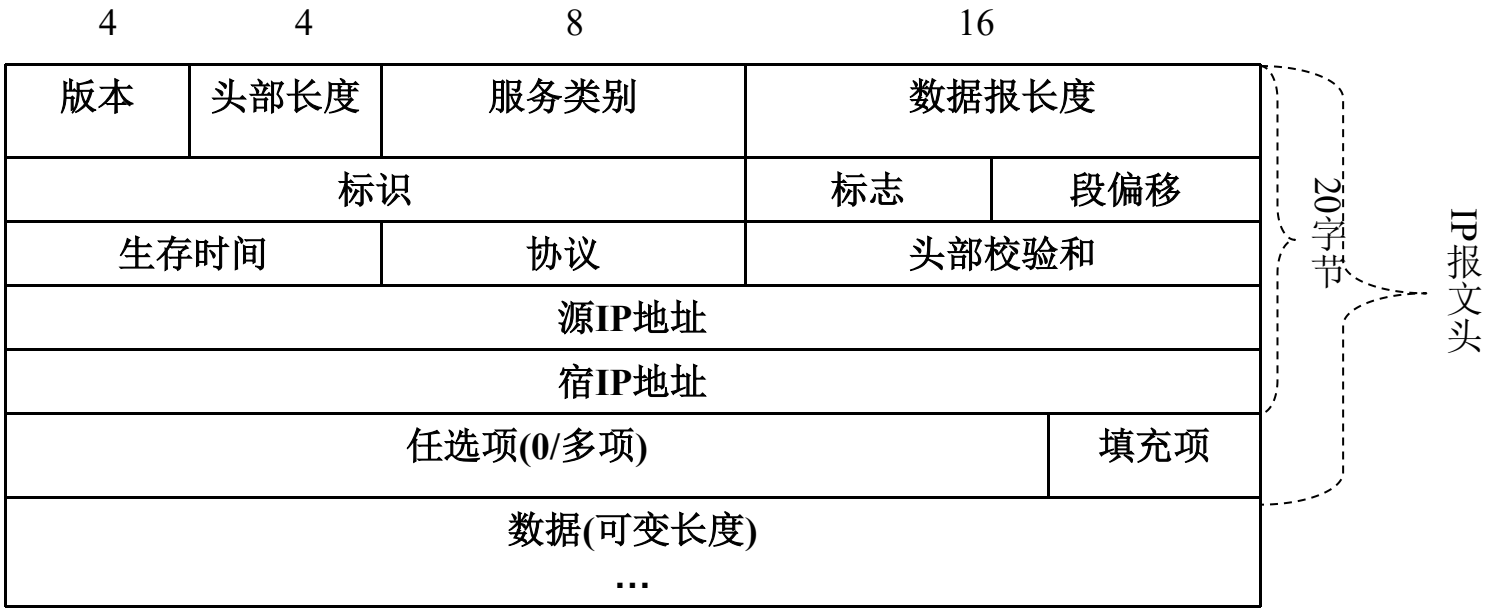
- TCP报文段的格式包括首部和数据。首部的前20个字节固定，后面有4N字节可选项。



TCP报文段格式


32位
0
8
16
24
31
源端口
目的端口
序号
确认号
数据偏移
保留
URG
ACK
PSH
RST
SYN
FIN
窗口
检验和
紧急指针
选项（长度可变）
填充
TCP首部
20字节固定部分
发送在前
TCP首部
TCP数据部分
IP首部
IP数据报的数据部分

回顾IP报文格式



- 上表所示是IP协议所规定的数据包格式
- 其中，数据项是运输层所交付的要传递的数据。
- 报文头是网络层为传递数据所加的各种控制信息，又称为数据包首部

TCP报文段格式（源端口、目的端口）



源端口和目的端口：各占 2 字，是运输层与应用层的服务接口。

TCP报文段格式（序号、确认号）



- 序号：4字节，指是本报文段所携带数据的第一个字节的序号：
  - 也称为报文段序号；
  - 范围 [0, $2^{32} - 1$]，序号增加到$2^{32} - 1$后，下一个序号回到0。
- 确认号：4字节，是期望收到对方的下一个报文段中数据的第一个字节的序号。

TCP报文段格式（数据偏移）



数据偏移（即首部长度）：占 4 位，它指出 TCP 报文段的数据起始处距离 TCP 报文段的起始处有多远。“数据偏移”的单位是 32 位字（以 4 字节为计算单位），最大60字节。

TCP报文段格式（紧急、确认）



紧急 URG：当 URG = 1 时，紧急指针字段有效，此报文段中有紧急数据，应尽快传送。

确认 ACK：只有当 ACK = 1 时确认号字段才有效。当 ACK = 0 时，确认号无效。

TCP报文段格式（推送、复位）



推送 PSH：收到 PSH = 1 的报文段，发送方立即发送，接收方尽快上交接收应用进程。

复位 RST：当 RST = 1 时，表明 TCP 连接中出现严重差错，必须释放连接，然后再重新建立运输连接。

TCP报文段格式（同步、终止）



同步 SYN：同步SYN = 1表示这是一个连接请求或连接接受报文。

终止 FIN：用来释放一个连接。FIN = 1 表明此报文段的发送端的数据已发送完毕，并要求释放运输连接。

TCP报文段格式（窗口）



- **窗口字段**：占 2 字节，让对方设置发送窗口的依据，单位为字节：
  - [0, $2^{16} - 1$] 之间的整数；
  - 告诉对方，从本报文段首部中的确认号算起，接收方目前允许对方发送的数据量；

TCP报文段格式（检验和）



检验和：占 2 字节。检验和字段检验的范围包括首部和数据这两部分。在计算检验和时，要在 TCP 报文段的前面加上 12 字节的伪首部。

TCP报文段格式（紧急指针）



- **紧急指针字段**：占 16 位，指出在本报文段中紧急数据共有多少个字节，放在本报文段数据的最前面（紧急数据的**最后一个字节**）：
  - URG = 1 时有效。

TCP报文段格式（选项）



选项字段：长度可变。TCP 最初只有一种选项，即最大报文段长度 MSS。MSS 告诉对方 TCP：“我的缓存所能接收的报文段的数据字段的最大长度是 MSS 个字节。”

MSS (Maximum Segment Size)：是 TCP 报文段中的数据字段的最大长度。数据字段加上 TCP 首部才等于整个的 TCP 报文段。

TCP报文段格式（填充）



填充字段：这是为了使整个首部长度是 4 字节的整数倍。

5.4.2 TCP服务可靠性实现



- TCP是基于不可靠无连接IP数据报服务之上的传输控制协议。
- 提供信源机与信宿机之间可靠的端到端的面向连接的服务。
  - 以“确认”和“超时重传”机制保证传输的可靠性。
  - 以流量控制来保证数据不溢出。
  - TCP传输还要以拥塞控制来保证网络不出现拥塞现象。

1、TCP连接可靠性实现



◊ 确认

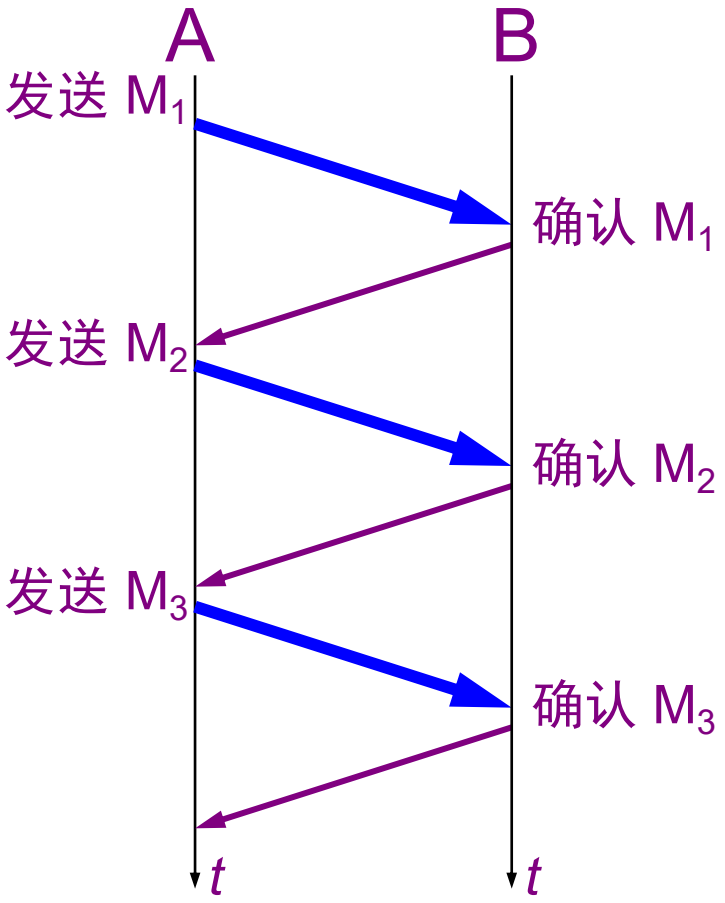
- □ 接收端收到数据单元，检查，确认无误后向发送端发出确认信息；
- □ 发送端收到确认信息后，才能删除本地保存的对应数据单元。

◊ 超时重传

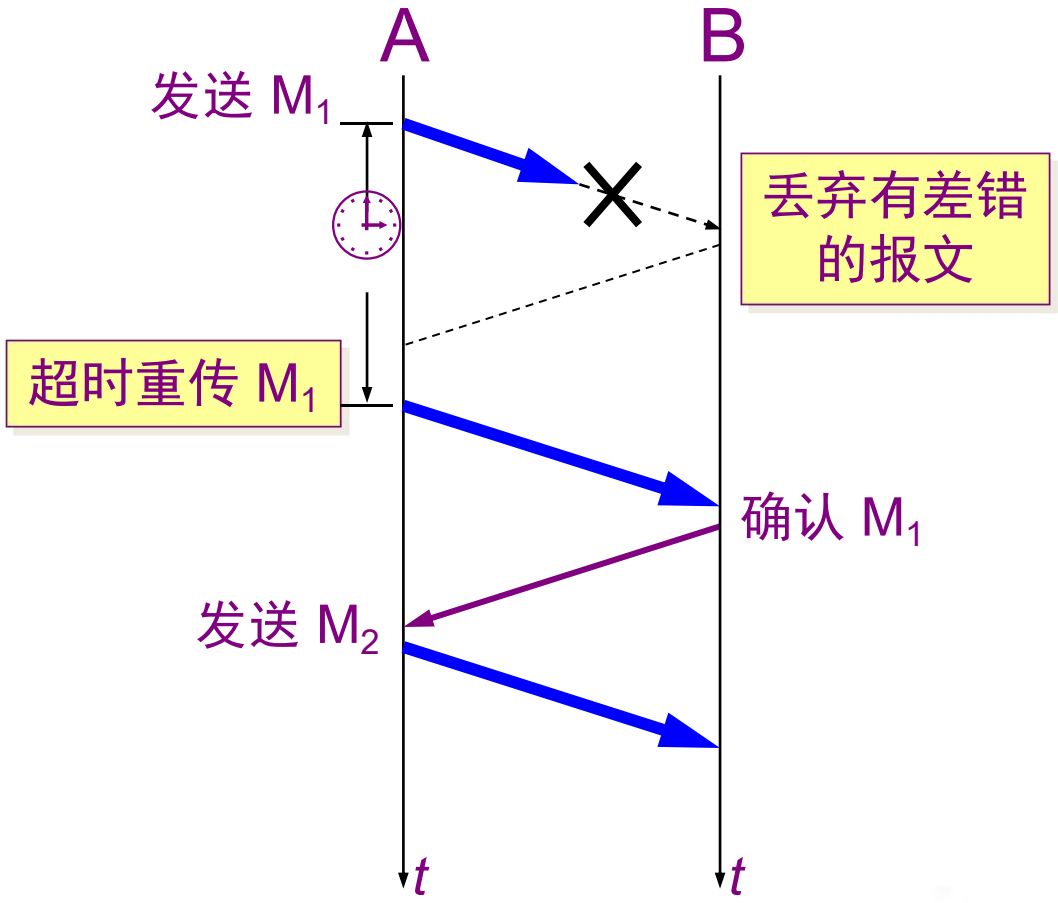
- □ 数据单元、确认信息皆有丢失的可能（因为网络层提供的IP数据报服务不可靠）
- □ 发送端发送一个数据单元以后，开始倒计时，若在规定的时间内未收到确认信息，就再次发送该数据单元

1、 TCP连接可靠性实现

◊ 超时重传



(a) 无差错情况



(b) 超时重传

2、流量控制----滑动窗口控制



- 为了提高数据发送效率，发送端一次发多个数据单元。发的越多速度越快。但要控制发送速度，避免接收端数据溢出。在发送端，采用滑动窗口进行流量控制。

- 滑动窗口的工作方式：
  - 数据单元在发送端排队，在队列上设置一个窗口，窗口内的数据单元可以发送，窗口外的数据单元不能发送。
  - 通过滑动窗口，将收到确认的数据单元移出窗口，并将新的被允许发送的数据单元移入窗口

- 滑动窗口的特点
  - 发送队列上的一个可以单向移动、宽度可变的窗口
  - 窗口宽度由接收端根据自己的接收能力确定

2、流量控制----滑动窗口控制

- A 收到了 B 的确认报文段：窗口值 20 字节，确认号为 31。
- 根据这两个数据，A 就构造出自己的发送窗口。



A 可以把落入发送窗口中的序号字节一次连续性全部发送出去：边发送边接收确认。

2、流量控制----滑动窗口控制



- 滑动窗口的宽度可变
  - 滑动窗口宽度由接收端根据自己接收缓存的剩余空间大小确定。
  - 接收端通过全双工TCP连接，在向发送端发出的报文中说明窗口宽度。
  - 滑动窗口宽度越大，允许发出的未经确认的数据越多，传输速度越快，反之越小。

2、流量控制----滑动窗口控制



3、拥塞控制----滑动窗口控制



- 当发生网络拥塞时，发送端要实时降低发送速度，减少网络的数据注入量。
- 发送端根据确认信息是否按时、按序抵达，间接检测网络是否发生拥塞，以及拥塞的程度。
- 当网络没有拥塞时，发送端逐步加大滑动窗口宽度；而当检测到网络拥塞时，发送端相应减小滑动窗口宽度。

- 流量控制与拥塞控制都是通过控制同一滑动窗口的宽度实现的。
- 滑动窗口实际宽度=min(rwnd, cwnd)。
  - rwnd由目的端根据自身空闲缓冲区大小确定
  - cwnd由发送端根据网络状况进行设置。

注：拥塞窗口(Congestion Window)cwnd是发送方根据自己估算的网络拥塞程度而设置的窗口值,反应网络的当前容量

TCP连接可靠性实现

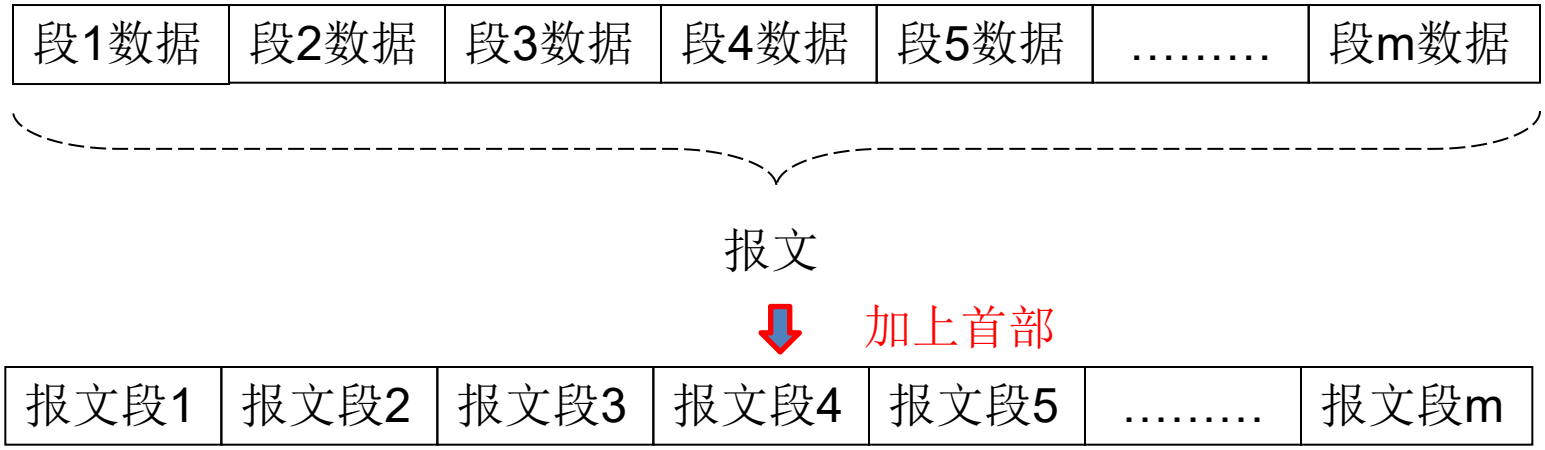


- “确认”、“超时重传”、流量控制、拥塞控制分别对应不同的控制机制
- TCP将它们统一地实现在自己的传输过程中。

5.4.3 TCP协议功能---TCP的编号与确认

◊ 1、TCP的编号与确认

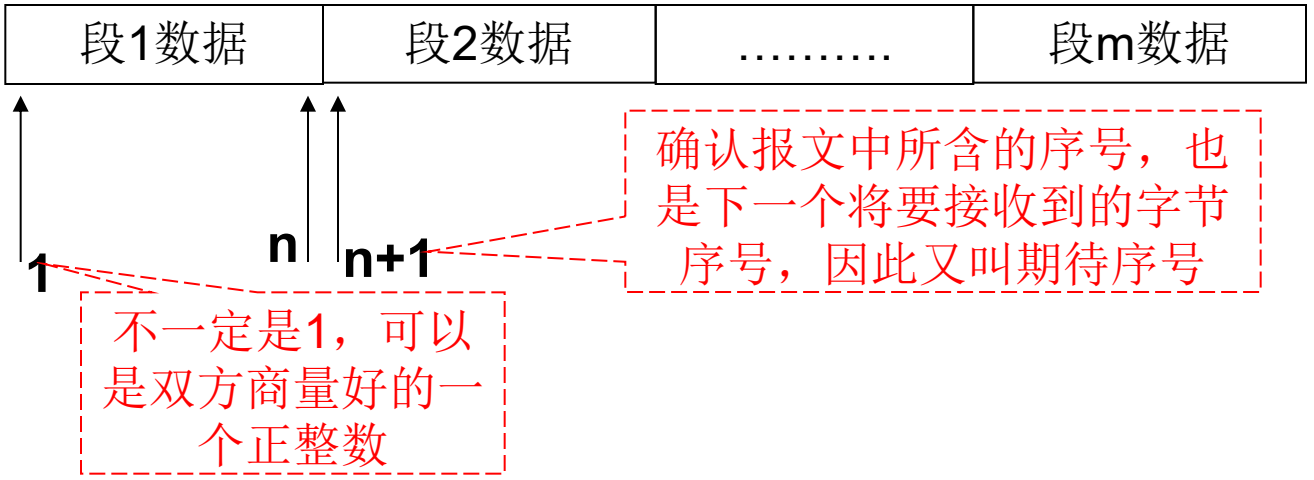
① TCP将所要传送的整个报文看作一个个字节组成的字节数据流，并对每一个字节编号；传输时，跟据滑动窗口的实际情况，将整个报文划分为若干数据段依次传输。



1、TCP的编号与确认

② TCP以报文段为单位进行传送，传送的报文段中，首字节的序号放在TCP首部的发送序号字段中。

③ 接收方收到一个数据长度为n的报文段并确认无误后，发回确认序号：发送序号+n



1、TCP的编号与确认

④ 如发送方在规定的时间内没有收到确认，就将未被确认的报文段重新发送。

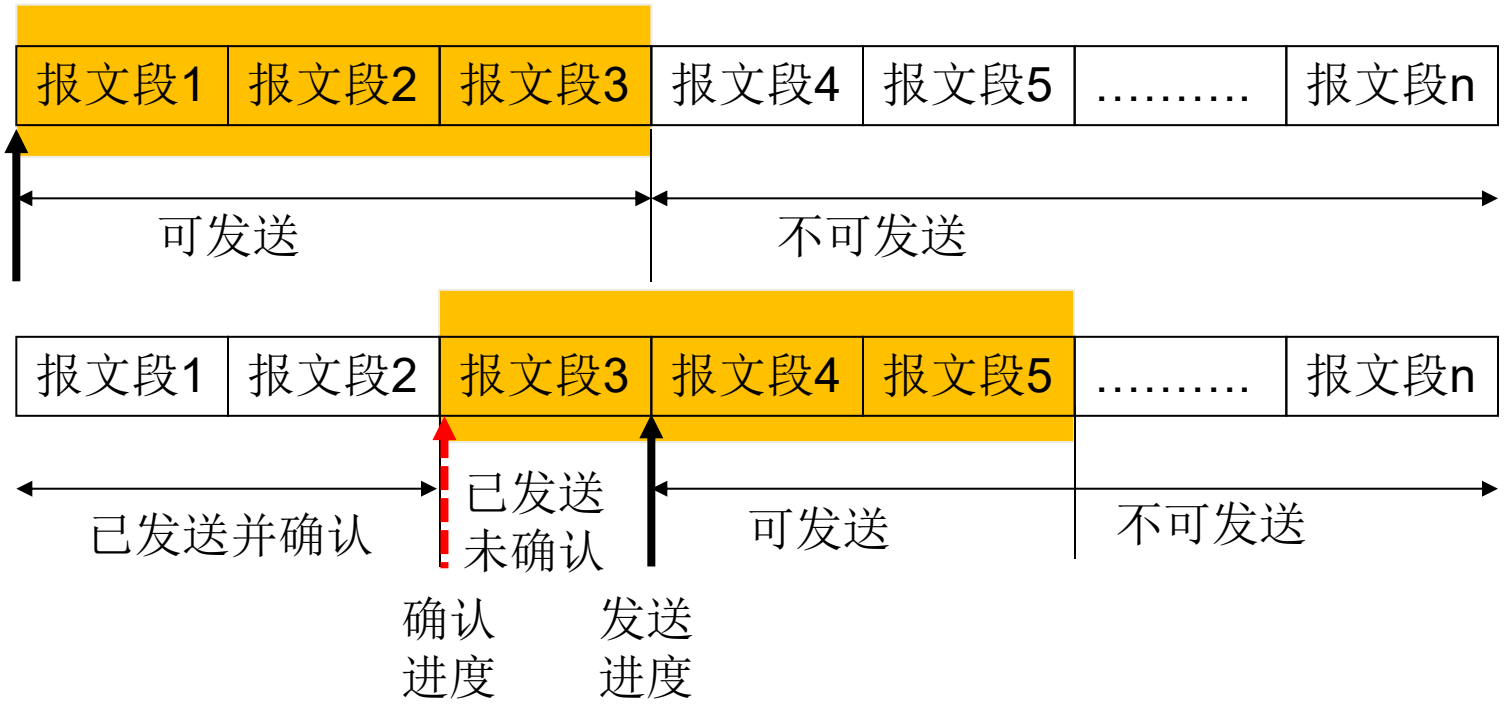
⑤ 接收方若收到有差错的报文段，则丢弃之；若收到重复的报文段，也将其丢弃，但要发回一个确认信息。

⑥ 如收到未按序号到达的报文段且无差错，存于接收缓冲区内，待所缺序号的报文段收齐后再一起上交给应用层，并发回按序号到达的最新报文段的确认信息。

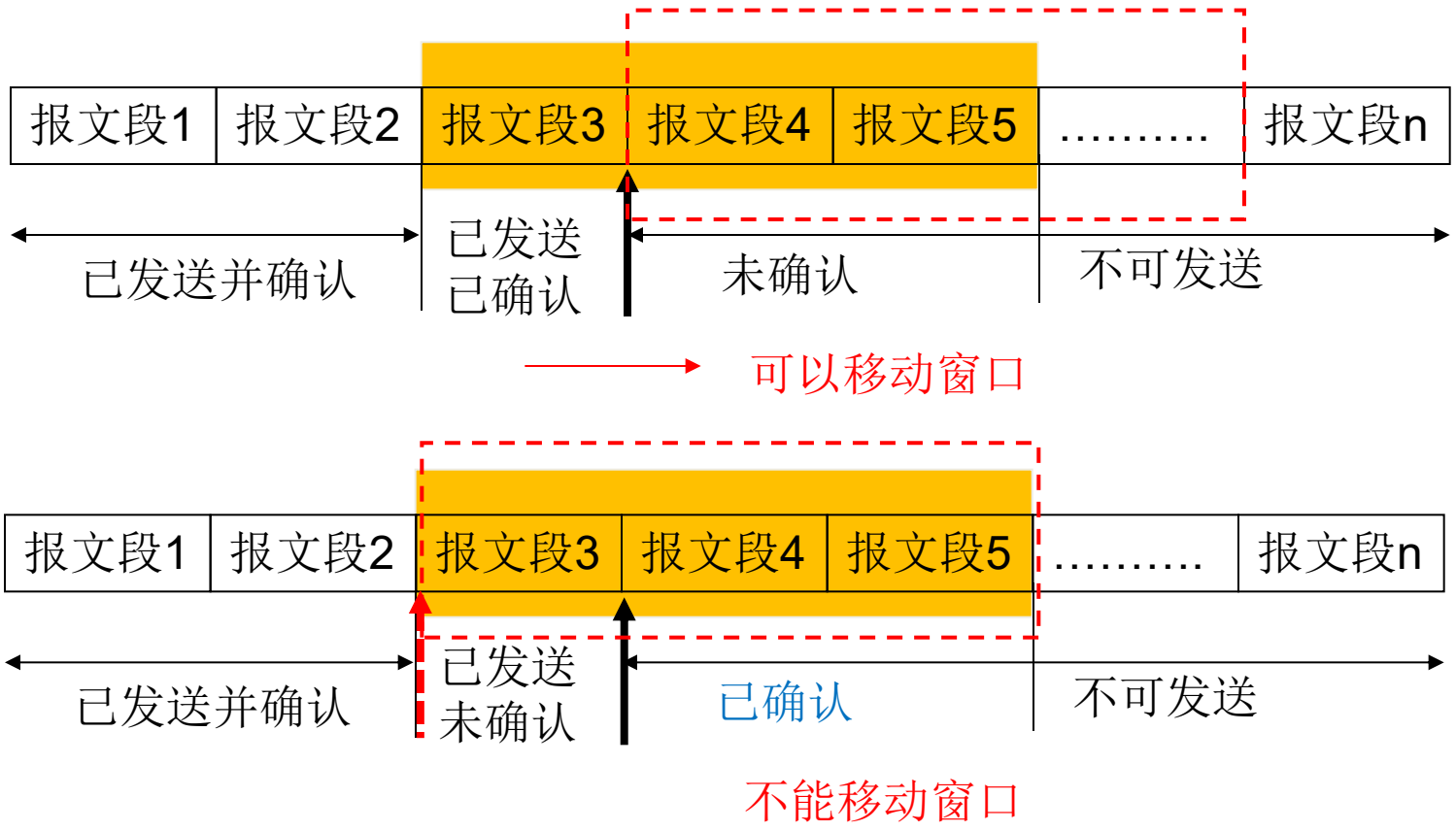
⑦ 接收方发回的确认序号是期待发送方下一次发送的段首字节号，又叫期待序号，意味着前面的字节都已收到。

⑧ 接收方不必为每一个报文段发确认信息。

1、TCP的编号与确认（发送端）

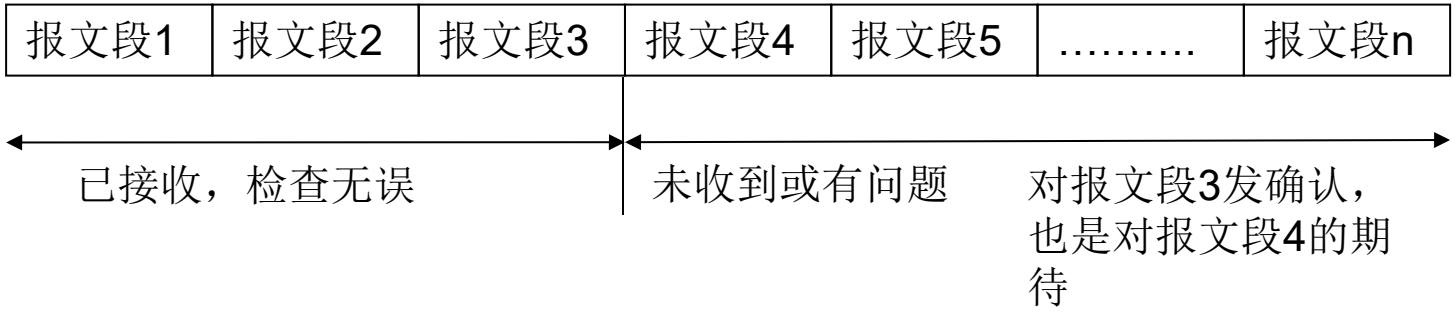


1、TCP的编号与确认（发送端）

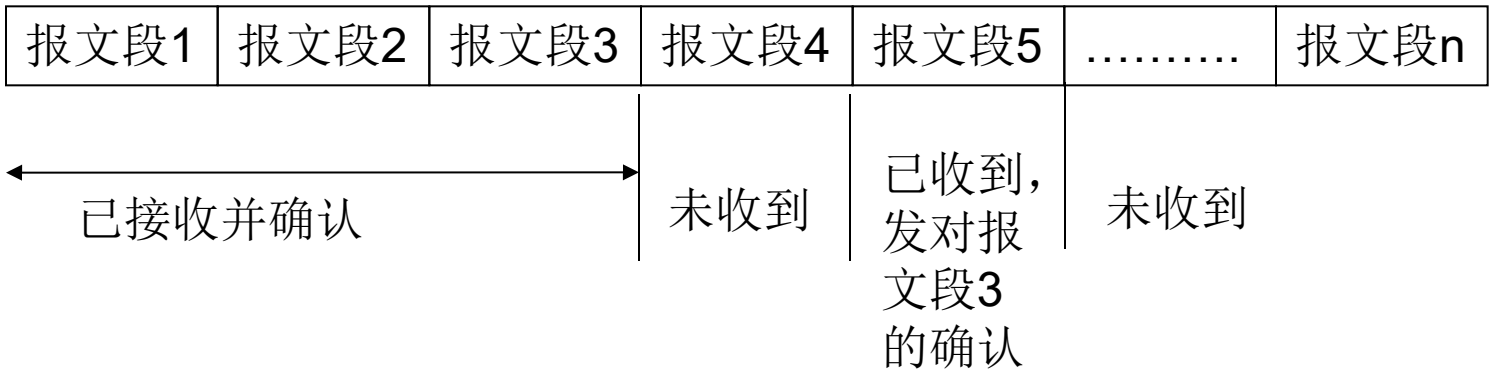


1、TCP的编号与确认（接收端）

- 对接收端而言，确认报文更确切地应该叫期待报文
- 接收端期待的是接收队列中最前面的未收到或已收到但有问题的那个报文段（图中报文段4）
- 如果同一批收到多个报文段（1，2，3），并检查确认无误，就应该期待报文段4了。这只需要对报文段3发确认。
- 因此，接收端不必为每一个数据单元发出确认，只需要根据接收队列情况，发相应的期待报文。
- 报文段3的确认意味着报文段3以前的数据单元均无问题（累积确认）



1、TCP的编号与确认（接收端）



- 迟迟未收到的数据单元会被多次期待

例题

◊ 计算题

主机A向主机B连续发出2个TCP报文段，其序号分别是70和100.

① 试求第一个报文段携带了多少字节的数据

② 主机B收到第一个报文段后发回的确认中，确认号是什么？

③ 如果B收到第二个报文段以后发回的确认号为180，试问第二个报文段有多少字节？

④ 如果第一个报文段丢失，第二个报文段被收到，B对第二个报文段的确认号是什么？

例题

◊ 解：

① 70和100分别是报文段1、2首字节的编号，因此报文段1所有字节的编号依次是70~99，报文段1共30字节。

② 确认序号又叫期待序号，是下一个报文段首字节编号，因此主机B收到报文段1后的确认序号为100.

③ 如果B的报文段2的确认序号为180，说明报文段3的首字符编号为180，也就是报文段2最后一个字节的编号为179，因此报文段2有179-100+1=80字节

④ 如果报文段1未收到，后续的确认序号都是对报文段1的期待，也就是确认序号为报文段1的首字节编号，70。

5.4.3 TCP协议功能---拥塞控制



- TCP的拥塞控制使用4种机制共同完成：
  - 慢开始
  - 拥塞避免
  - 快重传
  - 快恢复

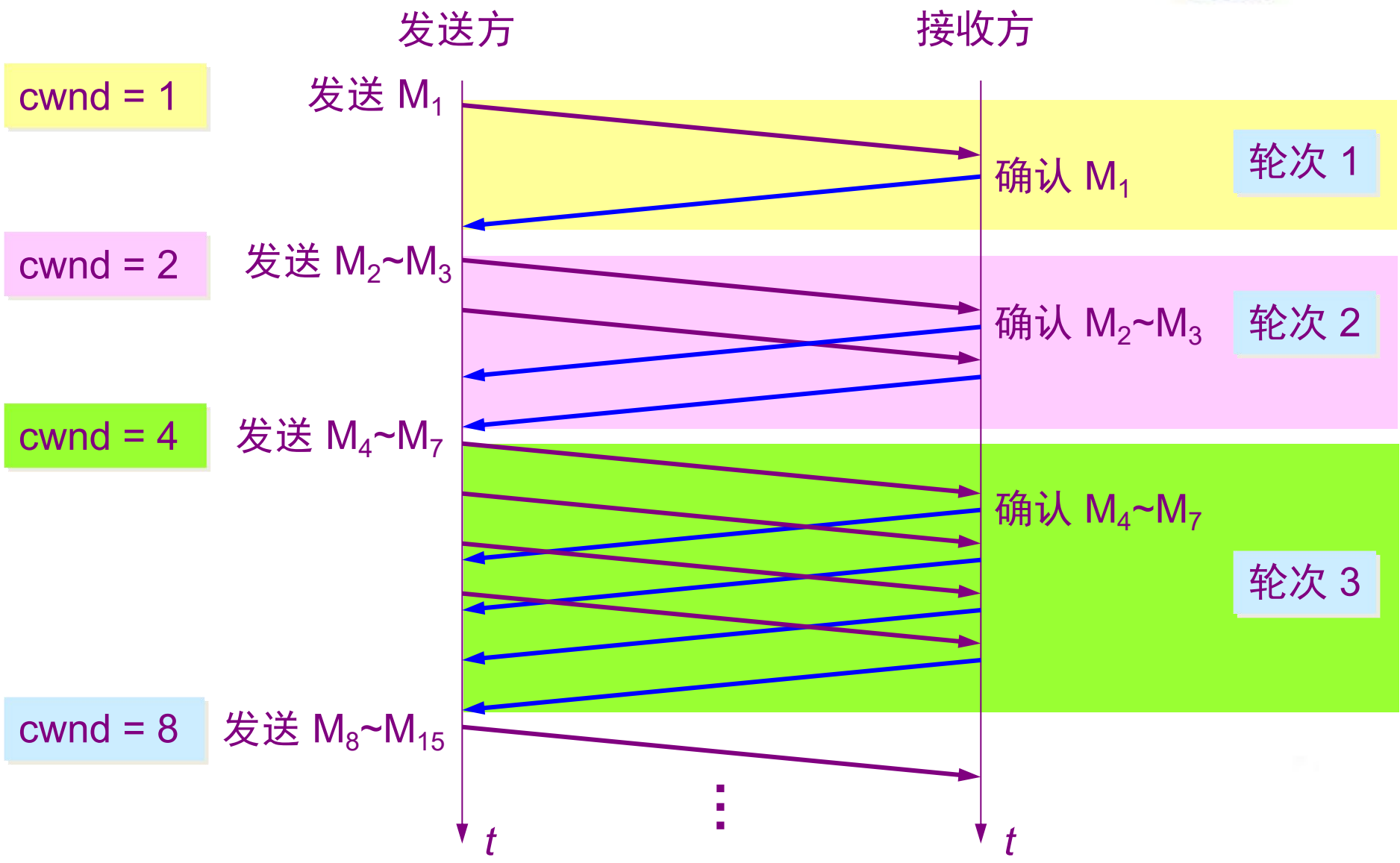
5.4.3 TCP协议功能---拥塞控制



◊ 慢开始

1. 开始时，设置cwnd=1，即只允许发送一个最大报文段长度（MSS），防止网络出现拥塞；
2. 发送方每收到一个新的确认报文后，将拥塞窗口加1
3. 等效于在一个往返周期（RTT）的传输轮次里，拥塞窗口被加倍了：cwnd = 2*cwnd。
4. 该阶段，传输速度以指数级数增加（2^x）

发送方每收到一个对新报文段的确认（重传的不算在内）就使 cwnd 加 1。



5.4.3 TCP协议功能---拥塞控制



◊ 拥塞避免

1. 当cwnd超过慢开始门限ssthresh（slow start thresh，另一个TCP的控制参数）时进入拥塞避免阶段；
2. 拥塞避免阶段cwnd线性增加，即每经过一个传输轮次（往返时间RTT），cwnd+1；
3. 该阶段，传输速度线性增加

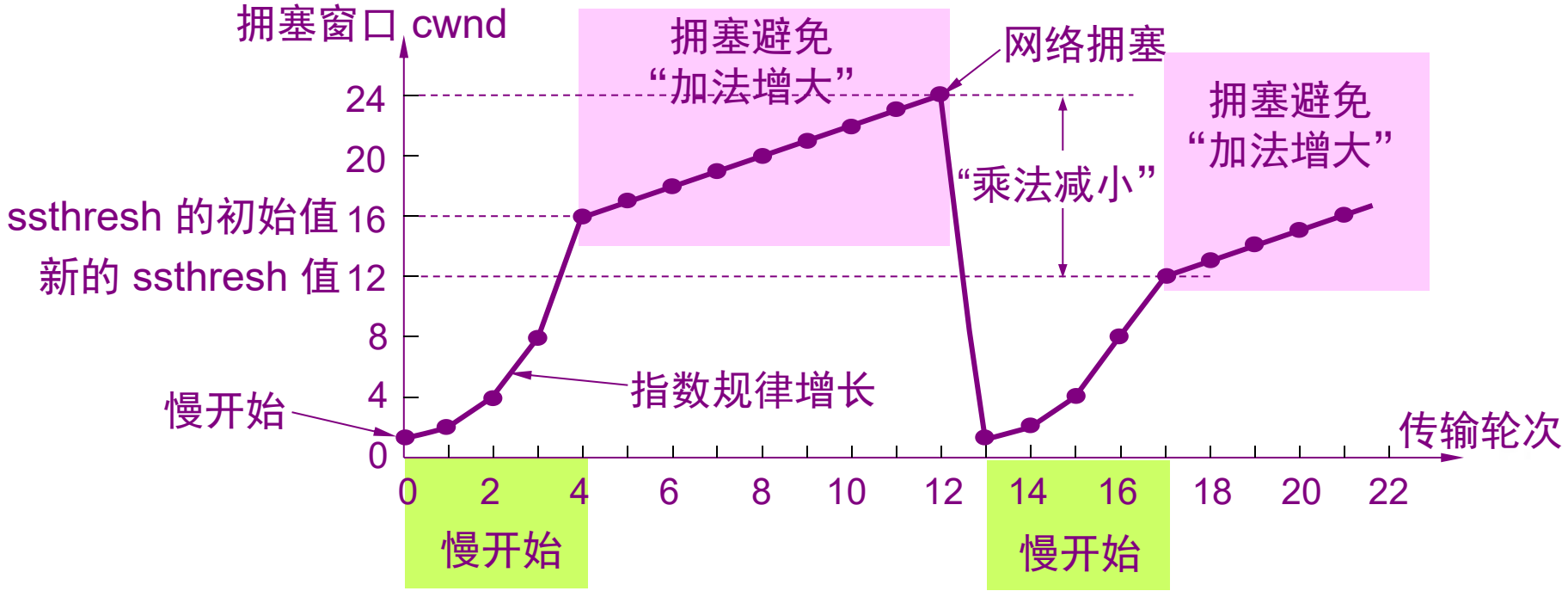
*实现方法：每收到一个新的ACK，cwnd增加1/cwnd，那么在RTT内，就会增加1，cwnd随着RTT呈线性增长

注：ACK （Acknowledge character）即是确认

5.4.3 TCP协议功能---拥塞控制

◊ 当检测到网络拥塞时， 超时重传

- 若在规定时间内没有收到确认报文，则判断网络发生拥塞
- 设置慢开始门限ssthresh为当前cwnd的一半
- 重置cwnd为1，重新进入慢开始状态（因为cwnd 小于ssthresh ）。
- 此刻，传输速度降到最低



5.4.3 TCP协议功能---拥塞控制

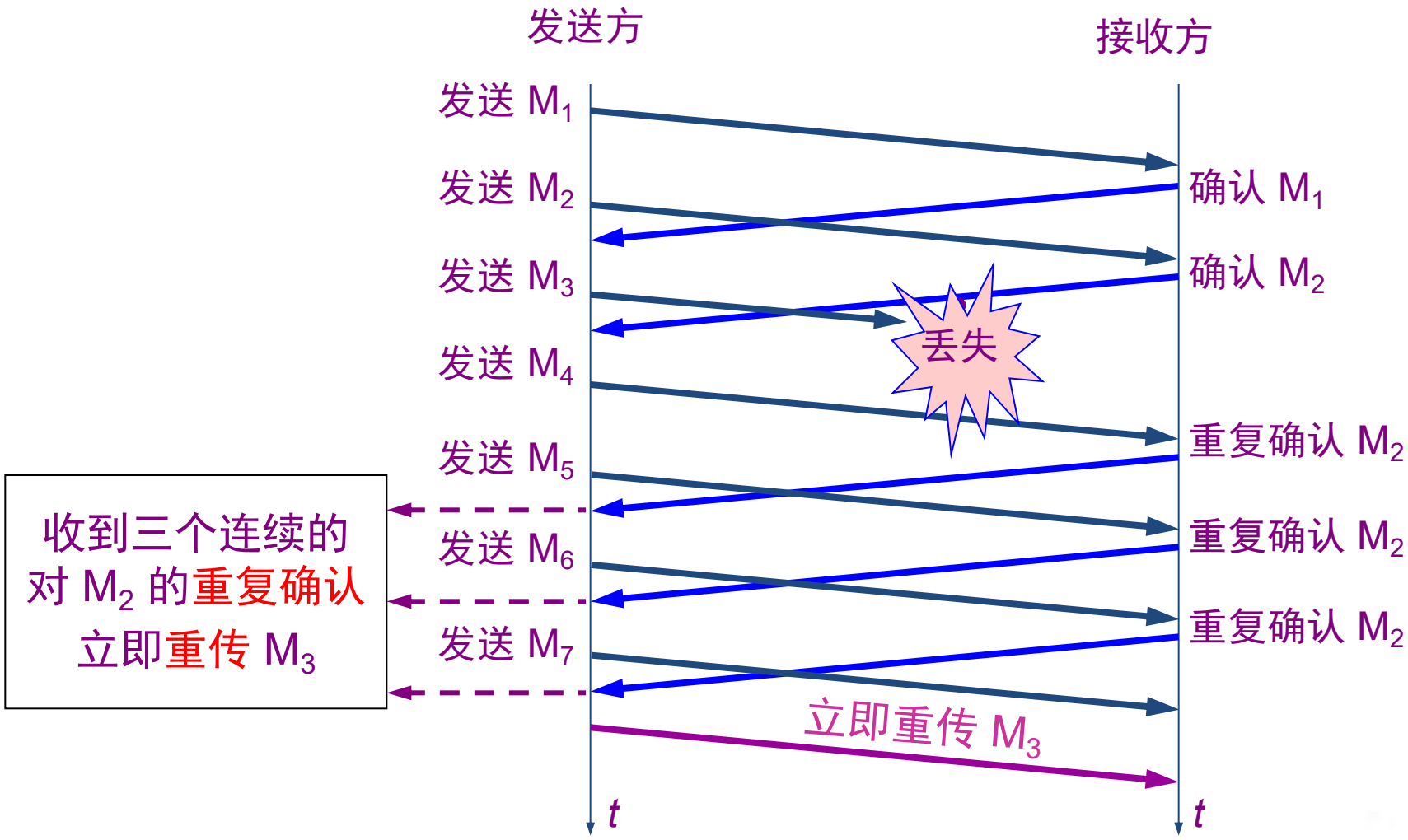


◊ 快重传

1. 如果中间单个报文段没有按序收到，例如接收端收到了1，2，4，5，6，7号报文段，则接收端在收到后面未按序到达的4，5，6报文段时，每次都只能发送2号确认报文（表示2号及以前的报文段已正确收到）；

2. 发送端在连续收到3个重复的2号确认报文后，可以断定有分组丢失，立即发送3号报文段

5.4.3 TCP协议功能---拥塞控制



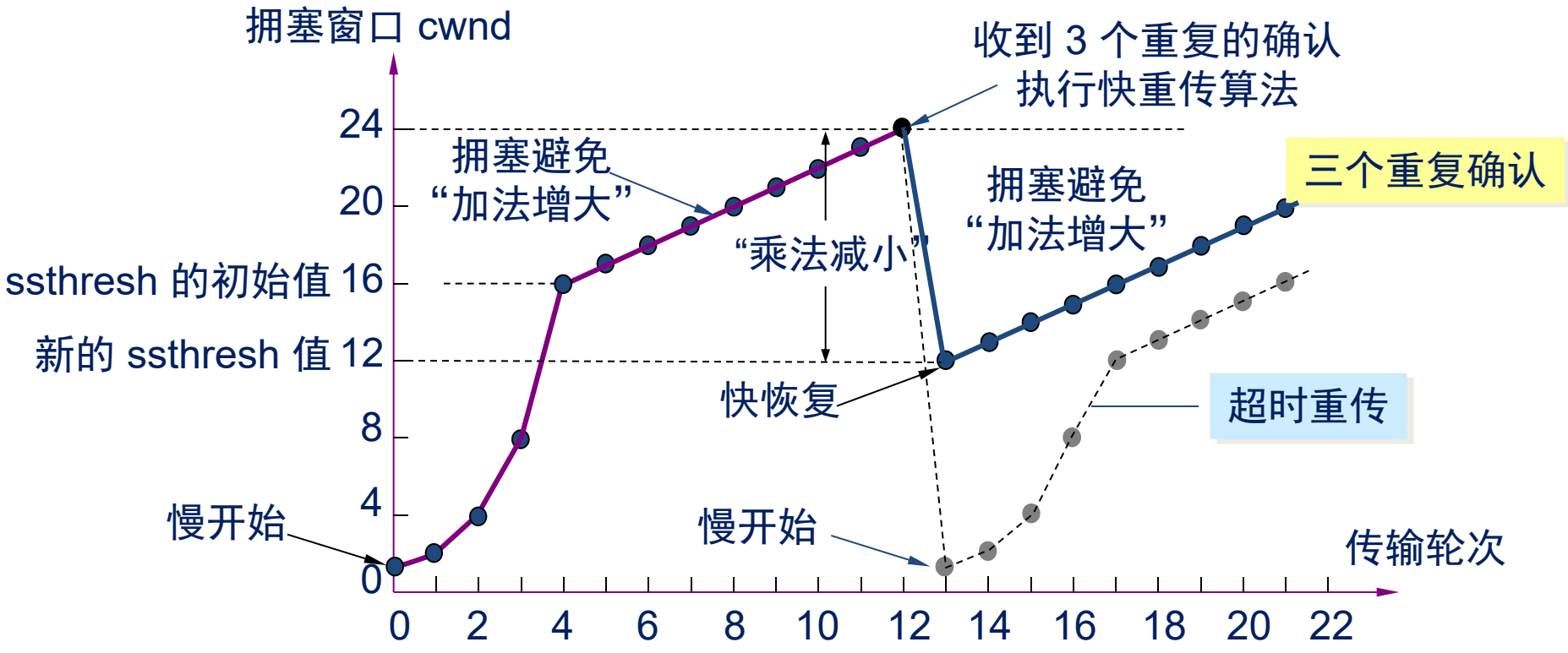
5.4.3 TCP协议功能---拥塞控制



- 快恢复
  - 在发送端收到三个重复确认报文时，将cwnd与ssthresh都为设置当前cwnd的一半
  - 由于cwnd与ssthresh值相同，所以后续按拥塞避免的机制运行

- 快恢复与超时重传的对比：
  - 相同之处：ssthresh都为设置当前cwnd的一半
  - 不同之处：超时重传cwnd设为1；快恢复cwnd设为当前cwnd的一半

从连续收到三个重复的确认转入拥塞避免



5.4.3 TCP协议功能---连接管理



- SYN：同步序列编号（Synchronize Sequence Numbers）。是TCP/IP建立连接时使用的握手信号。在客户机和服务器之间建立正常的TCP网络连接时，客户机首先发出一个SYN消息，服务器使用SYN+ACK应答表示接收到了这个消息，最后客户机再以ACK消息响应。这样在客户机和服务器之间才能建立起可靠的TCP连接，数据才可以在客户机和服务器之间传递。
- Sequence number (顺序号码)
- FIN ：终止（finish）

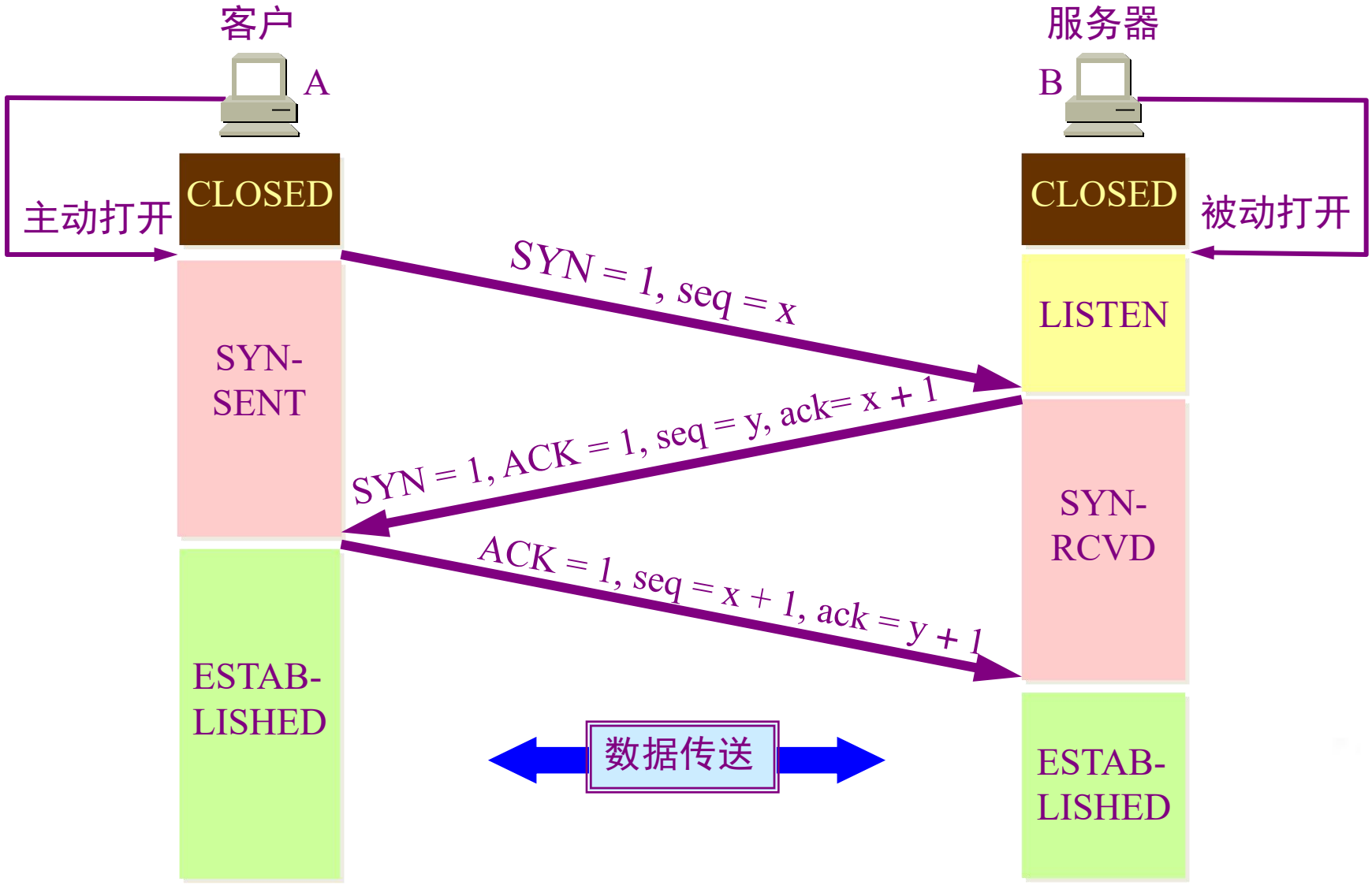
TCP的连接管理



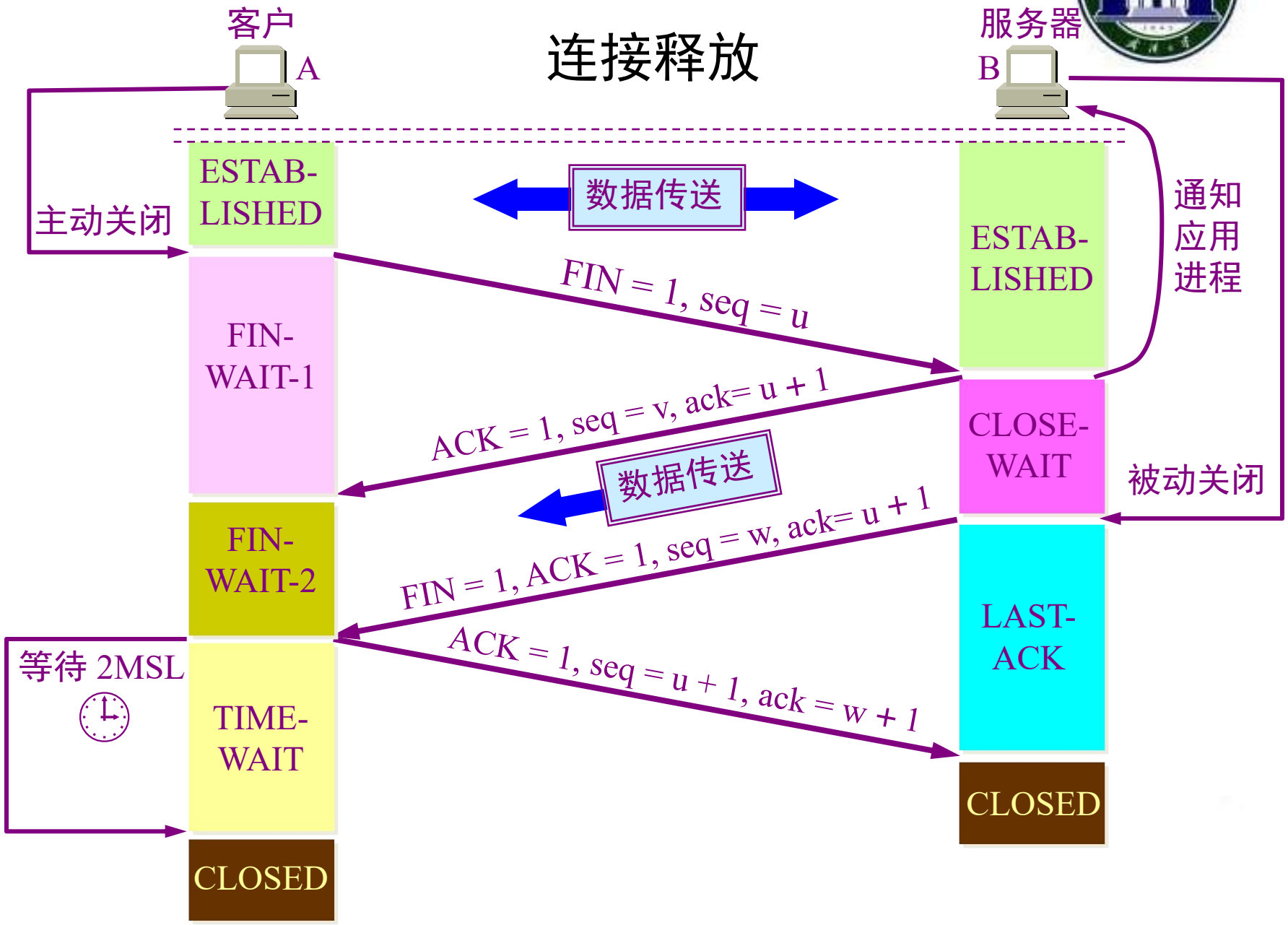
- 第一次握手：客户端发送位码为syn＝1，随机产生seq number= X 的数据包到服务器，服务器由客户端发送的SYN=1知道，客户端要求建立联机；
- 第二次握手：服务器收到请求后要确认联机信息，向客户端发送ack number=(客户端的seq+1)，syn=1，ack=1，随机产生seq=Y的包；
- 第三次握手：客户端收到后检查ack number是否正确，即第一次发送的seq number+1，以及位码ack是否为1，若正确，客户端会再发送ack number=(主机B的seq+1)，ack=1，服务器收到后确认收到的seq值与发送给客户端的seq+1是否相等和ack是否为1来判断连接是否建立成功。

TCP的连接管理

用三次握手建立 TCP 连接



TCP的连接管理


客户
A
连接释放
服务器
B
ESTAB-LISHED
数据传送
ESTAB-LISHED
主动关闭
通知应用进程
FIN = 1, seq = u
FIN-WAIT-1
ACK = 1, seq = v, ack= u + 1
CLOSE-WAIT
数据传送
被动关闭
FIN-WAIT-2
FIN = 1, ACK = 1, seq = w, ack= u + 1
LAST-ACK
等待 2MSL
ACK = 1, seq = u + 1, ack = w + 1
TIME-WAIT
CLOSED
CLOSED